

Environnement de Développement — VizHome

Guide de mise en place de l'environnement de développement local complet (backend + frontend) à partir des sources fournies.

Prérequis

Logiciels obligatoires

Outil	Version minimale	Lien
Docker Engine	24.0+	https://docs.docker.com/engine/install/
Docker Compose	v2.20+ (plugin)	Inclus avec Docker Desktop
Git	2.40+	https://git-scm.com/
Node.js	20 LTS (frontend uniquement)	https://nodejs.org/

Windows : utiliser [Docker Desktop](#) avec WSL 2 activé.

macOS : Docker Desktop suffit.

Linux : installer Docker Engine + Compose plugin via le gestionnaire de paquets.

Ressources machine recommandées

Ressource	Minimum	Recommandé
CPU	2 cœurs	4 cœurs
RAM	4 Go	8 Go
Disque	10 Go libres	20 Go libres

Structure des dépôts

Le projet VizHome est composé de **deux dépôts séparés** :

```
backend-vizhome/ ← API Django (ce dépôt)
frontend-vizhome/ ← Application Nuxt 4 (dépôt séparé)
```

Installation — Backend

1. Cloner le dépôt

```
git clone https://github.com/VizHome/backend-vizhome.git
cd backend-vizhome
```

2. Configurer les variables d'environnement

```
cp .env.example .env
```

Ouvrir `.env` et renseigner les champs essentiels pour le développement :

```
# Obligatoires en dev (les autres ont des valeurs par défaut)
DJANGO_SECRET_KEY=dev-secret-key-change-me-in-prod
POSTGRES_PASSWORD=vizhome_dev_password

# Optionnels – activer les fonctionnalités tierces (voir SETUP_KEYS.md)
# GEMINI_API_KEY=          ← génération IA (renders)
# STRIPE_TEST_SECRET_KEY= ← paiements
# GOOGLE_OAUTH_CLIENT_ID= ← connexion Google
# GITHUB_OAUTH_CLIENT_ID= ← connexion GitHub
```

Pour activer les clés API tierces, suivre le guide [SETUP_KEYS.md](#).

3. Démarrer la stack Docker

```
docker compose up -d
```

Cette commande démarre **7 services** :

Service	Port local	Rôle
api	8000	API Django (auto-reload)
celery	—	Worker de tâches asynchrones
postgres	5432	Base de données PostgreSQL 16
redis	6379	Cache + broker Celery
minio	9000 / 9001	Stockage S3 (API / Console)
pgweb	8081	Interface web Postgres
mailpit	8025	Capture d'emails de dev

4. Initialiser la base de données

```
# Appliquer les migrations
docker compose exec api python manage.py migrate

# Créer le superutilisateur admin
docker compose exec api python manage.py createsuperuser

# Bootstrap complet (migrations + Stripe + forum + i18n)
docker compose exec api python manage.py bootstrap
```

Raccourci : `bootstrap` orchestre toutes les initialisations en une seule commande.

5. Vérifier que tout fonctionne

URL	Service attendu
http://localhost:8000/health/ready	<code>{"status": "ok"}</code>
http://localhost:8000/api/docs/	Swagger UI
http://localhost:8000/admin/	Django admin

http://localhost:8081	pgweb (Postgres UI)
http://localhost:9001	MinIO Console
http://localhost:8025	Mailpit (emails)

```
# Vérification rapide en ligne de commande
curl http://localhost:8000/health/ready
```

Installation — Frontend

Le frontend (Nuxt 4) vit dans un dépôt séparé.

1. Cloner le dépôt frontend

```
cd ..
git clone https://github.com/VizHome/frontend-vizhome.git
cd frontend-vizhome
```

2. Configurer les variables d'environnement

```
cp .env.example .env
```

Variables clés :

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000/api/v1
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test... # optionnel
NEXT_PUBLIC_GOOGLE_CLIENT_ID=... # optionnel
```

3. Installer les dépendances et démarrer

```
npm install
npm run dev
```

Le frontend est accessible sur <http://localhost:3000>.

Commandes quotidiennes

Gestion de la stack backend

```
# Démarrer
docker compose up -d

# Arrêter
docker compose down

# Voir les logs en live
docker compose logs -f api # Django
docker compose logs -f celery # Worker async
```

```
docker compose logs -f postgres # Base de données

# Statut des services
docker compose ps
```

Django (toutes les commandes tournent dans le container `api`)

```
# Migrations
docker compose exec api python manage.py makemigrations
docker compose exec api python manage.py migrate

# Shell interactif
docker compose exec api python manage.py shell

# Bootstrap idempotent (migrations + setup complet)
docker compose exec api python manage.py bootstrap
docker compose exec api python manage.py bootstrap --skip-stripe
docker compose exec api python manage.py bootstrap --only migrate
```

Tests

```
# Tous les tests (~114 tests, ~30s)
docker compose exec api pytest

# Par application
docker compose exec api pytest apps/accounts
docker compose exec api pytest apps/renders

# Avec couverture
docker compose exec api pytest --cov=apps --cov-report=term-missing

# Arrêt au premier échec
docker compose exec api pytest -x

# Uniquement les tests échoués du run précédent
docker compose exec api pytest --lf
```

Qualité du code

```
# Lint (détection des erreurs)
docker compose exec api ruff check src/

# Formatage automatique
docker compose exec api ruff format src/

# Vérification des types (TypeScript-like pour Python)
docker compose exec api mypy src/
```

Outils de débogage

Inspecter PostgreSQL

```
# Shell Postgres direct
docker compose exec postgres psql -U vizhome -d vizhome

# Via pgweb (interface graphique)
# http://localhost:8081 – auto-connecté, aucun mot de passe nécessaire
```

Inspecter MinIO (stockage fichiers)

```
# Console web : http://localhost:9001
# Identifiants : vizhome / vizhome_minio_dev_password
```

Inspecter les emails de développement

Tous les emails Django (reset password, notifications, etc.) sont interceptés par Mailpit — **aucun email n'est envoyé sur Internet** en dev.

```
http://localhost:8025
```

Inspecter Redis (cache + files de tâches)

```
docker compose exec redis redis-cli
> KEYS *
> KEYS celery-task-*
```

Debugger avec ipdb

```
# Dans n'importe quel fichier Python
import ipdb; ipdb.set_trace()
```

Puis lancer le serveur de façon interactive (TTY requis) :

```
docker compose run --service-ports --rm api python manage.py runserver 0.0.0.0:8000
```

Tester l'API avec Bruno

[Bruno](#) est un client API alternatif à Postman. Une collection de 57 requêtes prête à l'emploi est disponible dans `bruno/` .

1. Installer Bruno : <https://www.usebruno.com/downloads>
2. **Open Collection** → sélectionner `backend-vizhome/bruno/`
3. Choisir l'environnement **Local**
4. Lancer `01-Health` > `Readiness` pour vérifier

Problèmes fréquents

Le container `api` ne démarre pas

```
docker compose logs api
# Chercher "Error" ou "Exception"
```

Cause la plus fréquente : `.env` mal configuré ou port 8000 déjà utilisé.

Les migrations échouent

```
docker compose exec api python manage.py migrate --run-syncdb
```

Rechargement de `.env` après modification

```
# `docker compose restart` NE relit PAS .env !
docker compose up -d --force-recreate api celery
```

Reset complet de la base (DESTRUCTIF — dev uniquement)

```
docker compose down -v
docker compose up -d
```

Mise à jour des dépendances Python

```
# Modifier src/requirements.txt puis :
docker compose build api celery
docker compose up -d --force-recreate api celery
```

Structure du projet backend

```
backend-vizhome/
├─ src/
│  ├─ config/          # Settings (base/dev/prod/test) + routing Celery
│  ├─ apps/
│  │  ├─ core/         # Healthchecks
│  │  ├─ accounts/     # Authentification, 2FA, OAuth, sessions
│  │  ├─ projects/     # Projets 3D, scènes, modèles importés
│  │  ├─ renders/      # Génération IA (Gemini, pipeline async)
│  │  ├─ gallery/      # Galerie publique des rendus
│  │  ├─ billing/       # Abonnements Stripe
│  │  ├─ forum/        # Forum communautaire
│  │  ├─ support/      # Ticketing helpdesk
│  │  ├─ admin_panel/  # Dashboard staff
│  │  ├─ contact/      # Formulaire de contact
│  │  └─ gdpr/         # Export et suppression de données (RGPD)
│  └─ manage.py
├─ docker/             # Dockerfiles (prod + dev)
├─ docker-compose.yml  # Stack de développement
├─ .env.example        # Variables d'environnement (modèle)
├─ pyproject.toml      # Config ruff, mypy, pytest
└─ docs/              # Documentation technique
```

Liens utiles

Ressource	URL
-----------	-----

API Swagger (dev)	http://localhost:8000/api/docs/
OpenAPI YAML	http://localhost:8000/api/schema/
Django Admin	http://localhost:8000/admin/
Architecture	docs/ARCHITECTURE.md
Déploiement prod	docs/DEPLOYMENT.md
Configuration clés API	SETUP_KEYS.md